

面向对象中的继承和多态

继承 (extends)

同类事物中，一般类的与特殊类的区别。

马：一般性的事物，表述范围更大，继承中的父类

白马：特殊性的事物，表述范围更小，继承中的子类

父类：super class，也会被翻译成“超类”

子类：subclass

```
public class 子类 extends 父类
{
    //...
}
```

子类会自动“获得”父类中定义的特征

子类可以编写自己“特殊”的特征

Java中继承的特点：

1. 一个子类，只能有一个直接的父类（单继承），但可以有多个间接父类
2. 一个类如果没有写明父类，它默认以Object类为父类，Object类是所有类的父类
也可以说，Object类是“引用数据类型”的意思

方法的重写 (override)

重写就是：子类重新改写父类的某个方法，子类与父类在同一个行为上，实现方式不一致；

语法要求：

1. 子类方法与父类方法同名，同参，同返回值类型
2. *子类方法的访问权限不低于父类的访问权限

3. 重写的其他名字：override，方法覆盖，运行时多态

父类中定义了一个方法

```
public void haha(int a, int b);
```

以下哪个方法不能定义在子类中

```
public void haha(int a); //重载
```

```
public void haha(int a, int b); //重写
```

```
public int xixi(int a, int b); //名字不一样
```

```
public int haha(int a, int b); //不行：方法名相同，既不是重载也不是重写，就不能定义
```

引用数据类型的类型转换

必须发生在继承或实现关系中

上溯造型（向上转型）

将子类实例转换成父类对象，自动转化

```
Bird b = new Bird();
```

```
Animal a = b;
```

```
Animal a = new Bird(); //上溯造型
```

对象类型	实例类型
------	------

将一个“鸟类”的个体当作“动物”看待

结果：失去“鸟类”的特殊特征，只剩“动物”特征

下溯造型

将一个父类对象强制转换成子类对象，前提该父类对象预先由该子类上溯造型。

子 -> 父 -> 子 鸟 -> 动物 -> 鸟

多态的实现（重点）

多态的实现可以分为四个步骤：这里以收费站对不同车辆收费的场景为例

1. 抽象出多种类的共同父类的概念；

轿车，客车，卡车 ----> 车辆

2. 子类**重写**父类的共同方法（支付pay方法）

轿车有轿车的pay实现，客车有客车的pay实现，卡车有卡车的pay实现

3. 借助**上溯造型**实现不同子类传入同一个业务方法

4. 基于重写：优先**调用**子类方法的特点，在业务方法中调用重写方法